SRS: Software Requirements Specification

Proyecto: WebApp de Visión Artificial, Detección Facial y Máscaras 3D en Meta Quest (WebXR)
Versión: 1.2.0 **Fecha:** 28 de octubre de 2026 **Estado:** Especificación técnica revisada contra fuentes. Los puntos marcados «**por verificar en visor**» no tienen fuente oficial y deben medirse en un Quest 3/3S real.

Historial de cambios

Versión	Cambios
1.1.0	Versión inicial con soporte Meta Quest Browser.
1.2.0	Referencias verificadas con URL. Se retira WebXR Raw Camera Access como mecanismo de captura (no implementado en Quest Browser). Se corrigen el hardware compatible (solo Quest 3/3S), los permisos ("Headset cameras"), la semántica de los dispositivos (la cámara *front* es la virtual del avatar), la resolución nativa (1280×960, 4:3), los landmarks (478, no 468), el uso de Hit Test (no aplica a rostros) y RNF-03 (`localhost` vía `adb reverse` no requiere HTTPS). Los requisitos de rendimiento pasan a ser objetivos medibles. Snippet de captura corregido.

---

1. Introducción

1.1 Propósito

Especificar los requisitos técnicos de una aplicación web inmersiva de visión artificial que:

- captura **una** de las cámaras passthrough frontales de **Meta Quest 3 / 3S** mediante `navigator.mediaDevices.getUserMedia` en Quest Browser (o una webcam en escritorio);
- detecta rostros localmente con **MediaPipe Face Landmarker**;
- proyecta máscaras 3D con three.js (opcionalmente React Three Fiber o Reactylon).

> **Corrección v1.2:** WebXR Raw Camera Access (`camera-access`) **no** es un mecanismo de captura disponible en Quest Browser. Una sesión `immersive-ar` que lo pide como requerido se rechaza con *"Feature 'camera-access' is not supported for mode: immersive-ar"* (Quest 3, Horizon OS v85) [R6]. Queda como mejora futura opcional, detrás de detección de capacidades.

1.2 Alcance del sistema

- Capturar el vídeo de una cámara passthrough (izquierda o derecha) del Quest 3/3S, o de una webcam tradicional.
- Enumerar las cámaras tras obtener el permiso y seleccionar por etiqueta `facing back` o por `facingMode: environment`, nunca por índice.
- Ejecutar la detección facial (**478 landmarks**, más la matriz de transformación facial 4×4) localmente en un Web Worker.
- Colocar y suavizar mallas GLB/GLTF o procedurales (máscaras o cascos) en el espacio 3D.

- Ofrecer un modo de depuración 2D y un modo `immersive-ar` con calibración manual de FOV y offsets.

Fuera de alcance (v1.2):

- captura estéreo simultánea (**por verificar en visor**);
- alineación por ojo usando intrínsecos de cámara (Quest Browser no los expone vía WebXR);
- dispositivos Quest 2 y Quest Pro.

1.3 Referencias verificadas

ID	Referencia	Notas de verificación	URL
R1	Reactylon – Camera Passthrough	Página marcada **EXPERIMENTAL**, basada en la investigación personal del autor, no en una especificación. Ejemplo con TensorFlow COCO-SSD, no MediaPipe. Indica que no hay acceso a frames crudos alineados ni a intrínsecos, y que el Hit Test con `entityTypes:['mesh']` no está soportado.	https://reactylon.com/docs/extended-reality/augmented-mixed-reality/camera-passthrough
R2	Foro Babylon.js – *WebXR camera passthrough support* (mismo autor)	Etiquetas observadas en otro build: `"camera 2 0, facing front"`, `"camera 2 1, facing back"`, `"camera 2 2, facing back"`.	https://forum.babylonjs.com/t/webxr-camera-passthrough-support/58274
R3	W3C Immersive Web – WebXR Raw Camera Access Module	*Draft Community Group Report* (11 dic 2025), no es una Recomendación del W3C. Diseño centrado en smartphones. Feature descriptor `camera-access`, `XRView.camera` y `XRWebGLBinding.getCameraImage()`. No es un polyfill.	https://immersive-web.github.io/raw-camera-access/
R4	meta-quest/webxr-showcases (MIT)	Sneaker Builder, RealMeasure, Chairs Etc y Flap Frenzy. Ninguno usa la cámara ni MediaPipe. El puerto `https://localhost:8081` es el de su servidor de desarrollo, no un requisito de plataforma.	https://github.com/meta-quest/webxr-showcases
R5	bandinopla/three-mediapipe-rig (MIT, npm `three-mediapipe-rig`)	Mueve un esqueleto (cara, cuerpo y manos) con la webcam. Peer deps `three@^0.182`, `@mediapipe/tasks-vision@^0.10.32`. No es específico de Quest.	https://github.com/bandinopla/three-mediapipe-rig
R6	Meta Community Forums – *Request: WebXR Raw Camera Access in Quest Browser*	`camera-access` no soportado en Quest Browser (Horizon OS v85). Chrome lo soporta desde la versión 107 (según el hilo).	https://communityforums.atmeta.com/discussions/Questions_Discussions/request-webxr-raw-camera-access-camera-access-feature-in-quest-browser/1367463
R7	Meta Help – permiso "Headset cameras" en Browser	Permiso del navegador. Solo un sitio a la vez. Al minimizar la ventana se detienen los frames. Menciona metadatos (focal, punto principal, dimensiones, posición de la cámara), pero sin API web documentada (**por verificar**). La cámara *virtual* nunca capta el entorno real.	https://www.meta.com/help/quest/1406582186767703/
R8	Meta – Passthrough Camera API (Native/Android)	Quest 3/3S, Horizon OS v74 o superior, passthrough activado. Permisos nativos `horizonos.permission.HEADSET_CAMERA` o `android.permission.CAMERA`.	https://developers.meta.com/vr/documentation/native/android/pca-native-overview/

ID	Referencia	Notas de verificación	URL
R9	Meta – Passthrough Camera API (Unity, rendimiento)	1280×960; 1280×1280 desde v83; 1728×1728 desde v209. Hasta 60 Hz. Latencia de 20–40 ms. Recomienda no depender de una resolución fija.	https://developers.meta.com/horizon/documentation/unity/unity-pca-overview/
R10	Meta – Investigación *Quest Browser Headset Camera Permission Issue*	Oculus Browser 150 / Chromium 150: no aparece el aviso de permiso, solo se enumera la cámara del avatar y `facingMode:{exact:'environment'}` da `OverconstrainedError`. Concediendo el permiso por ADB, se enumeran ambas cámaras passthrough con 1280×960 a 30 fps en `immersive-ar`.	https://developers.meta.com/vr/support/investigations/1557993751977106/
R11	Meta – IWSDK *Camera access*	Acceso a la cámara vía MediaDevices en sesiones XR. Recomienda 1280×720 a 30 fps por rendimiento.	https://developers.meta.com/vr/documentation/web/iwsdk-guide-camera-access/
R12	Meta Developers Blog – Passthrough Camera API v76	Publicación pública en v76; "browser support coming soon".	https://developers.meta.com/vr/blog/new-era-mixed-reality-passthrough-camera-api-machine-learning-computer-vision/
R13	GMO One Tech Blog (tercero)	Reporta que `getUserMedia` en el navegador funciona desde v77 (**no confirmado por Meta**).	https://tech.gmogshd.com/webxr-camera-api/
R14	Google AI Edge – Face Landmarker (Web)	478 landmarks 3D, 52 blendshapes opcionales y `facialTransformationMatrixes` 4×4.	https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/web_js
R15	google-ai-edge/mediapipe-samples-web – `face-landmarker.worker.ts`	Ejemplo oficial de Face Landmarker en un Web Worker con `ImageBitmap`.	https://github.com/google-ai-edge/mediapipe-samples-web/blob/main/src/workers/face-landmarker.worker.ts
R16	MediaPipe – `canonical_face_model.obj`	Modelo canónico (cm) usado para dimensionar la máscara procedural.	https://github.com/google-ai-edge/mediapipe/blob/master/mediapipe/modules/face_geometry/data/canonical_face_model.obj

---

2. Descripción general y arquitectura hardware/WebXR

2.1 Plataforma objetivo

Elemento	Valor	Fuente
Visores	Meta Quest 3 y Quest 3S (no Quest 2 ni Pro)	R8
Sistema	Horizon OS v74 o superior para la API nativa; navegador: v77 o superior según un tercero (**por verificar en visor**)	R8, R12, R13
Navegador	Meta Quest Browser, con el permiso **"Headset cameras"** concedido y el passthrough activado	R7, R8
Escritorio	Chrome/Edge/Firefox con webcam (modo de desarrollo y depuración)	—

2.2 Flujo de enumeración y captura en Quest Browser

```
+-------------------------------------------------------------------------+
|                            Meta Quest Browser                           |
|                                                                         |
|  0. getUserMedia({video:{facingMode:{ideal:'environment'}}})  -> permiso |
|     (sin permiso, enumerateDevices devuelve label/deviceId vacíos)      |
|                                                                         |
|  1. navigator.mediaDevices.enumerateDevices()  (etiquetas observadas, R1/R2)  |
|     ├── "camera 0, facing front" -> Cámara VIRTUAL del avatar (no ve el |
|     │                               entorno real, R7)                  |
|     ├── "camera 1, facing back"  -> Passthrough izquierda              |
|     └── "camera 2, facing back"  -> Passthrough derecha                |
|     * El formato de etiqueta varía entre builds ("camera 2 1, facing back").  |
|     * El orden NO está garantizado: filtrar por "facing back".         |
|                                                                         |
|  2. getUserMedia({video:{deviceId:{exact:id}, width:{ideal:1280},      |
|                          height:{ideal:960}, frameRate:{ideal:30}}})    |
|     └── MediaStream normal (sin pose ni intrínsecos asociados)         |
|                                                                         |
|  3. Inferencia y render                                                 |
|     ├── createImageBitmap(video) -> transfer -> Web Worker             |
|     │      MediaPipe Face Landmarker (VIDEO, GPU→CPU fallback)         |
|     └── three.js: pose = cabeza(t_captura) × offset_calibrado × matriz facial |
|         (sin Hit Test: no aplica a rostros)                             |
+-------------------------------------------------------------------------+
```

2.3 Restricciones conocidas

1. Sin alineación por ojo ni intrínsecos expuestos vía WebXR: la imagen de `getUserMedia` no está alineada con las vistas XR (R1, R6). La alineación en AR se logra con **calibración manual** (FOV y offset de la cámara respecto a la cabeza).
2. Solo un sitio puede usar la cámara a la vez, y al minimizar la ventana se detienen los frames (R7).
3. Captura estéreo simultánea (izquierda y derecha): **por verificar en visor**. La v1.2 usa una sola cámara (por defecto la derecha, como R1).
4. Continuidad de frames de `getUserMedia` durante `immersive-ar`: R10 reporta frames en `immersive-ar`; confirmar en el build objetivo (**por verificar en visor**).
5. La resolución nativa es 4:3 (1280×960), y existen resoluciones mayores en versiones recientes (R9). No depender de una resolución fija: leer `track.getSettings()`.

---

3. Requisitos específicos

3.1 Requisitos funcionales

Código	Descripción	Prioridad	Criterio de aceptación
RF-01.1	Solicitar el permiso de cámara **antes** de enumerar. Después ejecutar `enumerateDevices()` y seleccionar una cámara cuya etiqueta contenga `facing back`, ordenadas por número de cámara. Ojo configurable (izquierdo o derecho).	Alta	En Quest 3/3S con permiso concedido se obtiene un track `facing back`.
RF-01.2	Si no hay cámaras `facing back` (permiso "Headset cameras" denegado, dispositivo no compatible o escritorio), informar al usuario y usar la webcam (`facingMode: user`).	Alta	En escritorio la app arranca con webcam y muestra el aviso.
RF-01.3	Pedir la resolución con `ideal` (1280×960 a 30 fps en Quest) y, ante `OverconstrainedError`, reintentar con `facingMode: environment`. Registrar `track.getSettings()`.	Alta	Sin excepciones no controladas; la resolución real se muestra en la UI.
RF-01.4	Crear un elemento `<video>` (visible u oculto) como fuente del pipeline.	Alta	—
RF-02.1	Ejecutar MediaPipe Face Landmarker en modo `VIDEO` con `outputFacialTransformationMatrixes: true` (478 landmarks y matriz 4×4).	Alta	El overlay muestra 478 puntos.
RF-02.2	Ejecutar la inferencia en un **Web Worker** (module), transfiriendo `ImageBitmap`, con como máximo un frame en vuelo. Delegado GPU (OffscreenCanvas/WebGL2) con fallback a CPU. Si el worker falla, fallback al hilo principal cediendo el event loop.	Alta	El indicador de detector muestra `worker/GPU`, `worker/CPU` o `main-thread/*`.
RF-02.3	La captura se programa con temporizador o `requestVideoFrameCallback`, **no** con `window.requestAnimationFrame` (que no corre durante la sesión XR).	Alta	La inferencia continúa en `immersive-ar` (**por verificar en visor**).
RF-03.1	Aplicar a la máscara la pose (posición, rotación y escala) derivada de `facialTransformationMatrixes` (cm a m), con corrección de FOV (MediaPipe asume unos 63° vertical) y **suavizado** temporal (lerp/slerp exponencial, constante configurable).	Alta	La máscara sigue el rostro sin temblores visibles con suavizado de 80 ms.
RF-03.2	En `immersive-ar`, colocar la máscara como `pose_cabeza(t_captura) × offset_calibrado × matriz_facial`. Parámetros de calibración (FOV, offset XYZ, pitch/yaw, escala de profundidad y de máscara) editables y persistidos en `localStorage`. **No** usar WebXR Hit Test para la profundidad del rostro: solo choca con geometría del entorno, y Quest Browser no admite `entityTypes: ['mesh']` (R1).	Media	La máscara queda alineada tras calibrar (**por verificar en visor**).
RF-03.3	Soportar máscara procedural (sin assets) y carga de GLB/GLTF por URL.	Media	—

Código	Descripción	Prioridad	Criterio de aceptación
RF-04.1	Modo de depuración 2D: landmarks y caja sobre el vídeo.	Media	—
RF-04.2	Mostrar contadores de FPS de render y de inferencia y los ms por inferencia, también dentro de la sesión XR (HUD 3D).	Media	—
RF-05.1 *(futuro)*	Si `navigator.xr.isSessionSupported` y `camera-access` llegan a estar disponibles en Quest Browser, usar `XRWebGLBinding.getCameraImage()` como fuente alineada por vista (R3).	Baja	Detección de capacidades sin romper el flujo `getUserMedia`.

3.2 Requisitos no funcionales (NFR)

- **RNF-01 (Privacidad):** el vídeo se procesa al 100 % en el cliente (navegador del visor). No se transmite ningún fotograma ni landmark a servidores. El modelo y el WASM se sirven desde el mismo origen. Liberar la cámara (`track.stop()`) al salir.
- **RNF-02 (Rendimiento, objetivos medibles):**
  - Render XR: mantener la frecuencia configurada del visor (72/90 Hz). Medir con el contador de render o el OVR Metrics Tool. Objetivo: ≥ 95 % de frames a la frecuencia nominal (**por verificar en visor**).
  - Inferencia: **objetivo ≥ 30 fps** y ≤ 33 ms por inferencia en Quest 3/3S a 1280×960 o inferior. Hoy es solo un objetivo, sin medición publicada (**por verificar en visor**). Si no se alcanza, bajar la resolución de entrada o la frecuencia de inferencia y apoyarse en el suavizado.
  - La inferencia es asíncrona respecto al render: interpolar la pose entre resultados.
- **RNF-03 (Despliegue):** `getUserMedia` requiere contexto seguro.
  - Desarrollo: `adb reverse tcp:5173 tcp:5173` y abrir `http://localhost:5173` en el visor (`localhost` es contexto seguro; no requiere HTTPS).
  - Acceso por IP de red local o producción: **HTTPS** con un certificado válido.
- **RNF-04 (Compatibilidad):** detectar capacidades (MediaDevices, `immersive-ar`, Worker, OffscreenCanvas) y degradar con mensajes claros en lugar de fallar.

---

4. Snippet de referencia técnica: captura en Quest (corregido)

```
export type QuestEye = 'left' | 'right';

export async function getQuestPassthroughStream(eye: QuestEye = 'right'): Promise<MediaStream> {
  const md = navigator.mediaDevices;
  if (!window.isSecureContext || !md?.getUserMedia || !md.enumerateDevices) {
    throw new Error('Requiere contexto seguro (HTTPS o localhost) y MediaDevices');
  }

  // 1) Pedir permiso antes: sin él, label/deviceId llegan vacíos.
  const probe = await md.getUserMedia({ video: { facingMode: { ideal: 'environment' } }, audio: false
});
  probe.getTracks().forEach(t => t.stop());

  // 2) Filtrar por "facing back" (nunca por índice ni por un dígito suelto:
  //    "camera 2 0, facing front" también contiene '2').
  const cams = (await md.enumerateDevices()).filter(d => d.kind === 'videoinput');
  const back = cams.filter(d => /facing back/i.test(d.label));
  if (back.length === 0) {
    throw new Error('No hay cámaras passthrough: revisa el permiso "Headset cameras" y que sea Quest
3/3S');
  }
  const camNum = (l: string) => Number(/camera\s+(?:\d+\s+)?(\d+),/i.exec(l)?.[1] ?? NaN);
  back.sort((a, b) => camNum(a.label) - camNum(b.label)); // 1 = izq., 2 = der. (por verificar en
visor)
  const chosen = eye === 'left' ? back[0] : back[back.length - 1];

  // 3) Pedir 4:3 nativo (1280x960) con "ideal" para evitar OverconstrainedError.
  const video: MediaTrackConstraints = {
    deviceId: { exact: chosen.deviceId },
    width: { ideal: 1280 }, height: { ideal: 960 }, frameRate: { ideal: 30 },
  };
  try {
    return await md.getUserMedia({ video, audio: false });
  } catch (e) {
    if ((e as DOMException).name !== 'OverconstrainedError') throw e;
    return md.getUserMedia({ video: { facingMode: 'environment' }, audio: false });
  }
}
```

Errores del snippet v1.1 que se corrigen:

1. Enumeraba antes del permiso, cuando las etiquetas están vacías.
2. `label.includes('2')` coincide con la cámara *front* virtual en algunos builds.
3. `deviceId: undefined` combinado con `exact` era frágil.
4. Pedía 16:9 cuando la cámara es 4:3.
5. No tenía fallback ante `OverconstrainedError`.

---

5. Elementos pendientes de verificación en visor

#	Elemento	Cómo verificar
V1	Versión mínima de Horizon OS / Quest Browser con `getUserMedia` passthrough	Probar en v77+ y registrar versión del navegador.
V2	Correspondencia `camera 1` = izquierda, `camera 2` = derecha	Tapar una cámara física y observar el stream.
V3	Apertura simultánea de ambas cámaras	Dos `getUserMedia` con `deviceId` distintos.
V4	Continuidad de frames durante `immersive-ar`	Contador de inferencia en el HUD XR.
V5	Inferencia ≥ 30 fps (worker/GPU)	Contador de inferencia, a 1280×960 y a 640×480.
V6	FOV vertical real y offset de la cámara para la calibración	Calibrar con una persona a 1–2 m y guardar los valores.
V7	Exposición web de los metadatos de cámara mencionados en R7	Inspeccionar `track.getSettings()`/`getCapabilities()`.